

1. Purpose

This document defines what will be tested, how, and what result would support or refute the project's central claim, ahead of RTL implementation (Step 4) and experiments (Step 5). It builds directly on the Research Foundation Document (Deliverable 1) and the Baseline Controller Specification (Deliverable 2.5); both are referenced here, not repeated.

2. Novelty Statement

Relative to the three prior works that define this design space:

1. **Vs. Karthikeyan et al. (INCOFT 2025):** that work achieves workload-adaptive power management using K-means clustering and LSTM prediction across a multi-core system. This project achieves a comparable *qualitative* goal (adapting to workload character, not just idle duration) using a shift-register popcount and two comparators — zero ML hardware, sized for a single ALU-class block rather than a multi-core SoC.
2. **Vs. Agarwal et al. (ISQED 2006):** that work establishes multi-level sleep modes (our baseline directly implements their idle-cycle-count-only decision policy) but selects mode using idle-cycle count alone. This project adds a second, independent decision input ([activity_rate](#)) and adaptive hysteresis-band widening, directly targeting the thrashing failure mode that a single-condition static-threshold policy cannot avoid.
3. **Vs. Chang et al. (ISCAS 2012):** that work addresses *how* to wake up safely (two-phase rush-current shaping) but not *when* to sleep. This project's [sleep_controller.v](#) already incorporates their two-phase footer activation as the wake-up mechanism for both baseline and adaptive designs, so the comparison in this project is orthogonal to their contribution, not competing with it.

Patent-style claim sketch (per project brief requirement): *"A power-gating controller comprising: an activity monitor producing a rolling-window activity rate; a hysteresis predictor that gates sleep-mode entry on the conjunction of an idle-cycle count and the activity rate, and that widens its idle-cycle threshold when the activity rate exceeds a configurable limit, thereby suppressing sleep-mode entry during bursty operation without requiring per-cycle workload classification by external software or learned models."*

3. Baseline and Fair-Comparison Protocol

Defined in full in Deliverable 1 (Baseline Definition, Fair-Comparison Protocol) and Deliverable 2.5 (Baseline Controller Specification). Summary: single-module substitution ([hysteresis_predictor.v](#) → [fixed_threshold_predictor.v](#)), all other RTL/UPF/testbench held identical, both designs run at both design points (15 ns low-power, 8 ns high-speed corners).

4. Research Hypotheses

#	Hypothesis	Rationale	Falsified if...
H1	Under bursty traffic (Workload 1 — short, frequent idle gaps), the adaptive design produces fewer mode transitions than the baseline.	The baseline's single idle-cycle threshold has no defense against short gaps; the activity-rate gate should suppress entry that the baseline cannot.	Adaptive transition count $\geq$ baseline transition count on Workload 1.
H2	Under sparse/sustained-idle traffic (Workloads 2 and 5 — genuinely long idle), baseline and adaptive achieve statistically equivalent leakage savings.	Both designs should correctly enter Deep Sleep when idle is unambiguous; the adaptive design should not be cautious to the point of losing savings in the easy case.	Adaptive leakage saving is meaningfully lower than baseline's on these workloads.
H3	Under mixed traffic (Workload 4), the adaptive design achieves lower total energy (leakage + wake-up energy combined, not leakage alone) than the baseline.	Baseline thrashing under Workload 1-like sub-phases costs repeated wake-up energy that a leakage-only metric hides; H1 plus wake-up cost should compound into an energy win here specifically.	Baseline's total energy is equal or lower despite more transitions (would mean wake-up cost is negligible — also a useful, reportable finding either way).
H4	The adaptive design's area and critical-path overhead relative to the baseline is small (target: <10% of total controller area)	The "lightweight" claim in Deliverable 1's Expected Contributions depends on this; one comparator + one extra register stage vs. the	Post-synthesis area delta exceeds ~10%, or the adaptive design fails timing closure at either design point's clock constraint (8 ns / 15 ns)

		baseline's bare comparator should not cost much.	while the baseline meets it — either would undercut the "lightweight" framing and need to be reported as a limitation rather than a strength.
--	--	--	---

5. Experimental Design Matrix

Full factorial across designs $\times$ workloads $\times$ design points, all metrics collected per cell:

Dimension	Levels
Design	Baseline, Adaptive
Design point (clock corner)	Low-power (15 ns), High-speed (8 ns)
Workload	Burst, Sparse, Mixed, Sustained-idle (reuse existing testbench workloads 1/2/4/5; Workload 3, external wakeup, is a functional check, not a PPA comparison point)
Metrics per cell	Leakage power, dynamic power, area, Fmax/critical path, wake-up latency, mode-transition count, valid-result throughput

Area and Fmax/critical path are properties of the synthesized netlist — they depend on design $\times$ corner only (4 distinct values each), repeated across each design's 4 workload rows for table consistency. Leakage power, dynamic power, wake-up latency, mode-transition count, and throughput genuinely vary per workload and are measured across the full 16 cells.

This yields 2 designs $\times$ 2 corners $\times$ 4 workloads = 16 simulation/synthesis cells, each producing the metrics above.

Derived metric — total energy per workload run (used by H3, not directly reported by DC/ICC): computed as (leakage power $\times$ total simulated cycles) + (dynamic power $\times$ active-cycle fraction) + (mode_transitions $\times$ per-transition wake-up energy, estimated from wake-up latency $\times$ supply rail current during ramp). The exact formula will be finalized once real synthesis numbers exist in Step 5; this defines which raw columns it draws from.

H1 and H3 are read directly off this matrix (transition-count column, and the derived energy metric above); H2 is read off the leakage-power column on Workloads 2/5; H4 is read off the area and Fmax/critical-path columns at either corner (these shouldn't change much between corners for the same design).

6. Deferred to Later Steps

- Baseline RTL implementation: Step 4.
- Actual synthesis/P&R execution to populate the matrix: Step 5.
- Statistical treatment of H2's "equivalent" claim (e.g., what tolerance counts as equivalent) will be fixed once real numbers exist — premature to set a numeric threshold before any data is collected.